

第三章 Lesson6 Procedures

笔记整理自 2026 年 9 月 24 日课程

1. 过程调用的基本机制

过程 (procedure) 包括 C 语言中的函数、方法等抽象。一次过程调用必须协调三件事：

- **传递控制**：调用者跳到被调用者的入口；被调用者结束后回到调用点的下一条指令；
- **传递数据**：向被调用者传递参数，并把返回值交还调用者；
- **管理局部数据**：在执行期间为局部变量和临时值分配空间，返回时回收。

x86-64 并没有一条指令包办整个过程调用。call、ret、寄存器和栈只提供机制；参数放在哪些寄存器、哪些寄存器必须保存等选择由**应用二进制接口 (ABI)** 规定。编译器只使用当前函数实际需要的机制，例如没有局部数组的叶子函数可能完全不建立栈帧。

2. x86-64 运行时栈

2.1. 栈的方向与栈指针

运行时栈是一段按后进先出 (LIFO) 纪律管理的内存。x86-64 栈向**低地址**增长，%rsp 保存当前栈顶元素的地址，也就是已用栈区中的最低地址。

```

高地址    栈底 / 较早的栈帧
          ...
          返回地址
          保存的寄存器
%rsp ->   当前栈顶
低地址    尚未分配的空间
  
```

因此，“压栈”会减小 %rsp，“出栈”会增大 %rsp。栈中已经弹出的字节通常不会立即被清零；逻辑上的删除只是改变 %rsp，旧值之后可以被覆盖。

2.2. pushq 与 popq

pushq Src 把 8 字节操作数压入栈中，等价于：

```
subq $8, %rsp
movq Src, (%rsp)
```

硬件会先读取源操作数，再减小 %rsp 并写入栈顶。popq Dest 则从栈顶读出 8 字节，再移动栈指针：

```
movq (%rsp), Dest
addq $8, %rsp
```

这里的两段代码用于说明语义；真实的 pushq 和 popq 各是一条指令。若 Dest 是寄存器，popq 不会修改刚刚弹出位置的内存内容。

3. 控制传递：call 与 ret

3.1. 返回地址

call label 完成两步：把 **call 后一条指令的地址** 压入栈中，再把 %rip 改为 label。压入的地址称为返回地址。ret 做相反操作：从栈顶弹出一个地址并跳转到该地址。

```
call label:  R[%rsp] = return_address; %rsp -= 8; %rip = label
ret:        %rip = M[%rsp];           %rsp += 8
```

从效果上看，`call` 类似“压入返回地址后跳转”，`ret` 类似“弹出地址后间接跳转”。二者配合，使被调用者不需要事先知道调用者位于何处。

3.2. multstore 示例

```
long mult2(long a, long b) {
    return a * b;
}

void multstore(long x, long y, long *dest) {
    long t = mult2(x, y);
    *dest = t;
}

multstore:
    pushq %rbx           # 保存调用者的 rbx
    movq %rdx, %rbx      # 调用期间保存 dest
    call mult2           # 返回地址入栈，跳到 mult2
    movq %rax, (%rbx)     # *dest = 返回值
    popq %rbx            # 恢复调用者的 rbx
    ret

mult2:
    movq %rdi, %rax      # a
    imulq %rsi, %rax     # a * b
    ret
```

执行 `call mult2` 时，`%rsp` 减少 8，栈顶保存 `movq %rax, (%rbx)` 的地址；`mult2` 的 `ret` 取出该地址，控制流便回到 `multstore`。`%rbx` 的保存原因将在寄存器保存约定中说明。

4. 数据传递约定

4.1. 参数与返回值

x86-64 Linux ABI 优先使用寄存器传递整数和指针参数：

第 1 个	第 2 个	第 3 个	第 4 个	第 5 个	第 6 个	第 7 个及以后
<code>%rdi</code>	<code>%rsi</code>	<code>%rdx</code>	<code>%rcx</code>	<code>%r8</code>	<code>%r9</code>	调用者的栈帧

整数或指针返回值放在 `%rax` 中。超过 6 个的参数通过栈传递，由调用者在发出 `call` 前放置。浮点和向量参数使用另一组寄存器，本讲只讨论整数参数。

在 `multstore(x, y, dest)` 中，`x`、`y`、`dest` 分别位于 `%rdi`、`%rsi`、`%rdx`；调用 `mult2(x, y)` 时，前两个参数已经处于正确位置。`mult2` 把乘积留在 `%rax`，于是调用者能直接取得 `t`。

4.2. 参数寄存器也可被覆盖

参数寄存器属于 `caller-saved` 类别，被调用者可以把它们当作临时空间使用。因此 `multstore` 不能假设 `mult2` 返回后 `%rdx` 仍保存 `dest`，而要在调用前把它移到受保护的 `%rbx` 中。

5. 栈帧与局部数据

5.1. 为什么每次调用需要独立状态

递归函数可以同时存在多次尚未结束的调用实例。每个实例都要保留自己的返回地址、局部变量和保存的寄存器，因此这些状态按调用顺序组织成**栈帧 (stack frame)**。被调用者总先于调用者返回，正好符合 LIFO 次序。

典型栈帧从高地址到低地址包含：

组成	作用
栈上传入的参数	第 7 个及以后的参数，位于调用者为本次调用准备的区域
返回地址	call 自动压入，ret 自动取出
旧 %rbp	使用帧指针时保存；现代编译器常省略帧指针
保存的寄存器	保护必须跨调用保持的寄存器值
局部变量和临时空间	放置不能只保存在寄存器中的值，例如取地址的局部变量
参数构造区	为下一次调用准备栈上传递的参数

进入函数时的“序言”通常通过 `pushq` 和 `subq` 分配空间；返回前的“尾声”用 `addq` 和 `popq` 恢复状态。`%rbp` 可作为稳定的帧指针，但不是每个栈帧都必须使用它。

5.2. 局部变量示例：call_incr

```
long incr(long *p, long val) {
    long x = *p;
    *p = x + val;
    return x;
}
```

```
long call_incr(void) {
    long v1 = 15213;
    long v2 = incr(&v1, 3000);
    return v1 + v2;
}
```

```
incr:
    movq (%rdi), %rax    # x = *p, 同时准备返回 x
    addq %rax, %rsi      # y = x + val
    movq %rsi, (%rdi)    # *p = y
    ret
```

```
call_incr:
    subq $16, %rsp      # 为局部数据分配 16 字节
    movq $15213, 8(%rsp) # v1 = 15213
    movl $3000, %esi     # 第 2 个参数 val
    leaq 8(%rsp), %rdi   # 第 1 个参数 &v1
    call incr
    addq 8(%rsp), %rax    # v2 + 更新后的 v1
    addq $16, %rsp       # 回收局部空间
    ret
```

`v1` 必须放入内存，因为程序要取得它的地址。`leaq 8(%rsp), %rdi` 只计算该局部变量的地址，不读取其值。`movl $3000, %esi` 比对应的 64 位立即数编码更短，且写 32 位寄存器会自动清零高 32 位，所以能正确传递正数 3000。

6. 寄存器保存约定

若调用者在寄存器中保存一个仍要使用的值，被调用者又覆盖同一寄存器，程序就会出错。ABI 用两类约定解决这个问题：

- **caller-saved**：调用者若需要跨越一次调用保留该值，就在调用前自行保存；被调用者可自由覆盖；
- **callee-saved**：被调用者若想使用该寄存器，必须先保存原值，并在返回前恢复。

类别	寄存器	含义
返回值 / caller-saved	%rax	被调用者可覆盖；整数返回值放在这里
参数 / caller-saved	%rdi, %rsi, %rdx, %rcx, %r8, %r9	传入参数后可被被调用者覆盖
临时 / caller-saved	%r10, %r11	调用者负责保护仍需使用的值
callee-saved	%rbx, %rbp, %r12-%r15	被调用者使用前保存，返回前恢复
栈指针	%rsp	返回时必须恢复为调用前约定的值

6.1. callee-saved 示例

```

long call_incr2(long x) {
    long v1 = 15213;
    long v2 = incr(&v1, 3000);
    return x + v2;
}

call_incr2:
    pushq %rbx                # 保存调用者原来的 rbx
    subq $16, %rsp
    movq %rdi, %rbx           # 用 callee-saved 寄存器跨调用保存 x
    movq $15213, 8(%rsp)
    movl $3000, %esi
    leaq 8(%rsp), %rdi
    call incr
    addq %rbx, %rax            # x + v2
    addq $16, %rsp
    popq %rbx                 # 恢复进入本函数前的 rbx
    ret

```

call_incr2 自己是 incr 的调用者，同时也是上层函数的被调用者。它选择 %rbx 保存 x，是因为 %rbx 能跨越 call incr；但这也使它必须通过开头的 pushq 和结尾的 popq 履行 callee-saved 责任。

7. 递归如何使用栈

递归不需要特殊的机器指令。每次递归调用都会产生新的返回地址和栈帧，保存该层自己的局部状态；寄存器保存约定保证内层调用不会破坏外层尚需使用的值。

```

long pcount_r(unsigned long x) {
    if (x == 0)
        return 0;
    return (x & 1) + pcount_r(x >> 1);
}

pcount_r:
    movl $0, %eax             # 先准备终止情形的返回值 0
    testq %rdi, %rdi
    je .L6                    # x == 0 时直接返回
    pushq %rbx                # 保存调用者的 rbx
    movq %rdi, %rbx
    andl $1, %ebx              # 本层需要保留 x & 1
    shrq %rdi                  # 递归参数 x >> 1
    call pcount_r
    addq %rbx, %rax            # 本层最低位 + 递归结果
    popq %rbx

```

.L6:
ret

非终止层把 $x \& 1$ 存在 `%rbx`，因为递归调用会覆盖参数寄存器和 `caller-saved` 寄存器。每层都把原 `%rbx` 压栈，因此内层返回后能够恢复外层的值。终止层不使用 `%rbx`，所以可以直接返回而不建立额外栈帧。相同机制也支持互递归：只要调用和返回仍遵循 LIFO，P 调 Q、Q 再调 P 也不需要新的规则。

8. 小结

- x86-64 栈向低地址增长，`%rsp` 指向栈顶；`pushq` 先减 8 再写入，`popq` 读出后加 8。
- `call` 压入下一条指令的地址并跳转；`ret` 弹出返回地址并跳回。
- 前 6 个整数或指针参数依次放在 `%rdi`、`%rsi`、`%rdx`、`%rcx`、`%r8`、`%r9`，返回值放在 `%rax`。
- 每次调用用栈帧保存返回信息、寄存器和局部数据；只有确有需要时才分配栈空间。
- `caller-saved` 寄存器由调用者保护，`callee-saved` 寄存器由使用它的被调用者保存并恢复。
- 普通调用约定天然支持递归：每层调用都拥有独立的返回地址和局部状态。